

## Scenario 01

### Search Functionality

"Test the search feature."

**write down what information is missing from it.**

- It does not mention background like application, module name
- Role is missing
- Output format
- Tone is missing (standard)
- Examples are missing
- Not mentioned manual, automation, performance testing

### **Output:**

I searched for "**search functionality test**" and retrieved multiple relevant results, including:

- **Test cases for search functionality** (Zoho QEngine) – covers positive/negative test cases, autocomplete, filters, edge cases, and performance.
- **How to Test Search Functionality in Mobile and Web Apps** (SUSATest) – provides a practical checklist for query input, search results, and usability.
- **50 Search Feature Test Cases** (Virtuoso QA) – includes scenarios for exact match, partial match, autocomplete, filters, and result validation.

- **BrowserStack: Test Cases for Search Functionality** – covers positive/negative testing, UI, performance, security, accessibility, and localization.

### ICEPOT Prompt:

As a Senior QA Engineer, I need test cases for the Search Functionality of the "XYZ" application. Include both 15 Positive and Negative test cases, exclude all non-functional test cases and it should follow standard QA ,the output should be in an Excel (xlsx) format with parameters of S. No, Test Case ID, Test Steps, Test Data, Environment, Browser & Version, Expected Result, Remarks (blank)

### Output:

| S. No | Test Case ID | Test Steps                                                                           | Test Data     | Environment | Browser & Version | Expected Result                        | Remarks |
|-------|--------------|--------------------------------------------------------------------------------------|---------------|-------------|-------------------|----------------------------------------|---------|
| 1     | SRCH_001     | Launch the application, enter a valid keyword in the Search field, and click Search. | Valid keyword | QA          | Chrome 137        | Relevant search results are displayed. |         |

| S. No | Test Case ID | Test Steps                                       | Test Data          | Environment | Browsers & Version | Expected Results                                        | Remarks |
|-------|--------------|--------------------------------------------------|--------------------|-------------|--------------------|---------------------------------------------------------|---------|
| 2     | SRCH_002     | Search using an exact keyword.                   | Existing item name | QA          | Chrome 137         | Exact matching result is displayed.                     |         |
| 3     | SRCH_003     | Search using a partial keyword.                  | Partial text       | QA          | Chrome 137         | Matching results containing the keyword are displayed.  |         |
| 4     | SRCH_004     | Search using uppercase and lowercase characters. | pRoDuCt            | QA          | Chrome 137         | Search is case-insensitive and returns correct results. |         |

| S. No | Test Case ID | Test Steps                               | Test Data  | Environment | Browsers & Versions |         | Remarks                                                              |
|-------|--------------|------------------------------------------|------------|-------------|---------------------|---------|----------------------------------------------------------------------|
|       |              |                                          |            |             | Browser             | Version |                                                                      |
| 5     | SRCH_005     | Search with leading and trailing spaces. | " Laptop " | QA          | Chrome              | 137     | Spaces are ignored and relevant results are displayed.               |
| 6     | SRCH_006     | Search using numeric values.             | 12345      | QA          | Chrome              | 137     | Relevant numeric results or "No results found" message is displayed. |
| 7     | SRCH_007     | Search using alphanum                    | ABC123     | QA          | Chrome              | 137     | Matching results are                                                 |

| S.<br>N<br>o | Test<br>Case ID | Test Steps                                                        | Test<br>Data | Environm<br>ent | Brows<br>er &    Expecte |                                                                                      | Remar<br>ks |
|--------------|-----------------|-------------------------------------------------------------------|--------------|-----------------|--------------------------|--------------------------------------------------------------------------------------|-------------|
|              |                 |                                                                   |              |                 | Versio<br>n              | d Result                                                                             |             |
|              |                 | eric<br>values.                                                   |              |                 |                          | displaye<br>d.                                                                       |             |
| 8            | SRCH_0<br>08    | Clear the<br>search<br>field and<br>perform<br>another<br>search. | Phone        | QA              | Chro<br>me<br>137        | Previous<br>results<br>are<br>cleared<br>and new<br>results<br>are<br>displaye<br>d. |             |
| 9            | SRCH_0<br>09    | Click<br>Search<br>without<br>entering<br>any value.              | Blank        | QA              | Chro<br>me<br>137        | Validatio<br>n<br>message<br>or<br>default<br>behavior<br>is<br>displaye<br>d.       |             |

| S. No | Test Case ID | Test Steps                            | Test Data | Environment | Browsers & Version | Expected Result                                          | Remarks |
|-------|--------------|---------------------------------------|-----------|-------------|--------------------|----------------------------------------------------------|---------|
| 10    | SRCH_010     | Search for a non-existing keyword.    | XYZ987654 | QA          | Chrome 137         | "No results found" message is displayed.                 |         |
| 11    | SRCH_011     | Search using only " " spaces.         | " "       | QA          | Chrome 137         | Validation message is displayed; search is not executed. |         |
| 12    | SRCH_012     | Search using only special characters. | @#\$%^&*  | QA          | Chrome 137         | Application handles the input gracefully.                |         |

| S. No | Test Case ID | Test Steps                            | Test Data             | Environment | Browsers & Version | Expected Result                                                   | Remarks |
|-------|--------------|---------------------------------------|-----------------------|-------------|--------------------|-------------------------------------------------------------------|---------|
|       |              |                                       |                       |             |                    | y without errors.                                                 |         |
| 13    | SRCH_013     | Search using unsupported characters . | <>/                   | QA          | Chrome 137         | Appropriate validation or no results message is displayed.        |         |
| 14    | SRCH_014     | Search using a very long keyword.     | 256+ character string | QA          | Chrome 137         | Input is handled correctly ; appropriate results or validation is |         |

| S. No | Test Case ID | Test Steps           | Test Data                                                                         | Environment | Browser & Version |  | Expected Result | Remarks                                                    |
|-------|--------------|----------------------|-----------------------------------------------------------------------------------|-------------|-------------------|--|-----------------|------------------------------------------------------------|
|       |              |                      |                                                                                   |             |                   |  |                 |                                                            |
| 15    | SRCH_015     | Search using emojis. |  | QA          | Chrome 137        |  | displayed.      | Application handles the input gracefully without crashing. |

**Compare both outputs side by side and write 3 observations on what improved and why.**

1. Weak prompts generate generic outputs, whereas the ICEPOT framework provides application-specific and tester-focused outputs.
2. Weak prompts provide only high-level information, while the ICEPOT framework generates detailed and comprehensive test cases that are useful for software testing.
3. The ICEPOT framework generates test cases that are easy to understand, well-structured, and highly useful for testing activities.



-----

-----

-----

-----

-----

-----

-----

## Scenario 02

### File Upload

X

"Generate test cases for file upload."

### Output:

#### 1. Functional Test Cases

| TC ID  | Test Scenario                   | Expected Result               |
|--------|---------------------------------|-------------------------------|
| FU_001 | Upload a valid file             | File uploads successfully     |
| FU_002 | Upload multiple valid files     | All files upload successfully |
| FU_003 | Upload file using drag-and-drop | File uploads successfully     |
| FU_004 | Upload file using Browse button | File uploads successfully     |
| FU_005 | Replace an uploaded file        | New file replaces old file    |

| TC ID  | Test Scenario                                 | Expected Result                       |
|--------|-----------------------------------------------|---------------------------------------|
| FU_006 | Remove uploaded file before submission        | File is removed successfully          |
| FU_007 | Upload file and submit form                   | File is stored and linked correctly   |
| FU_008 | Cancel upload in progress                     | Upload stops successfully             |
| FU_009 | Retry failed upload                           | Upload resumes successfully           |
| FU_010 | Upload file after page refresh (if supported) | Upload resumes or behaves as designed |

---

## 2. File Type Validation

| TC ID  | Test Scenario                          | Expected Result         |
|--------|----------------------------------------|-------------------------|
| FU_011 | Upload supported file type (PDF)       | Upload successful       |
| FU_012 | Upload supported image file (JPG, PNG) | Upload successful       |
| FU_013 | Upload supported document (DOCX, XLSX) | Upload successful       |
| FU_014 | Upload unsupported file type (.exe)    | Error message displayed |

| TC ID  | Test Scenario                        | Expected Result                          |
|--------|--------------------------------------|------------------------------------------|
| FU_015 | Upload unsupported archive format    | Upload rejected                          |
| FU_016 | Upload file with incorrect extension | Upload rejected                          |
| FU_017 | Rename .exe file to .jpg and upload  | Validation detects actual file type      |
| FU_018 | Upload file without extension        | Handled as per requirements              |
| FU_019 | Upload corrupted file                | Appropriate validation/error shown       |
| FU_020 | Upload password-protected file       | System handles according to requirements |

---

### 3. File Size Validation

| TC ID  | Test Scenario                     | Expected Result          |
|--------|-----------------------------------|--------------------------|
| FU_021 | Upload file below maximum limit   | Upload successful        |
| FU_022 | Upload file exactly at size limit | Upload successful        |
| FU_023 | Upload file exceeding size limit  | Error displayed          |
| FU_024 | Upload zero-byte file             | Validation message shown |

| <b>TC ID</b> | <b>Test Scenario</b>                             | <b>Expected Result</b>          |
|--------------|--------------------------------------------------|---------------------------------|
| FU_025       | Upload very large file                           | Upload handled per requirements |
| FU_026       | Upload multiple files exceeding total size limit | Validation message displayed    |

---

#### 4. File Name Validation

| <b>TC ID</b> | <b>Test Scenario</b>                     | <b>Expected Result</b>              |
|--------------|------------------------------------------|-------------------------------------|
| FU_027       | Upload file with normal name             | Upload successful                   |
| FU_028       | Upload file with spaces in name          | Upload successful                   |
| FU_029       | Upload file with special characters      | Handled correctly                   |
| FU_030       | Upload file with Unicode characters      | Upload successful                   |
| FU_031       | Upload file with very long filename      | Validation performed                |
| FU_032       | Upload duplicate filename                | Rename or overwrite per requirement |
| FU_033       | Upload file with leading/trailing spaces | Handled correctly                   |

---

## 5. Security Test Cases

| TC ID  | Test Scenario                                         | Expected Result             |
|--------|-------------------------------------------------------|-----------------------------|
| FU_034 | Upload executable file (.exe)                         | Rejected                    |
| FU_035 | Upload script file (.js)                              | Rejected if restricted      |
| FU_036 | Upload malicious file containing malware              | Malware detection triggered |
| FU_037 | Upload file containing virus                          | Upload blocked              |
| FU_038 | Upload file with SQL injection in filename            | No system impact            |
| FU_039 | Upload file with XSS payload in filename              | Payload sanitized           |
| FU_040 | Upload file with path traversal characters (../)      | Rejected/sanitized          |
| FU_041 | Upload double-extension file (file.jpg.exe)           | Rejected                    |
| FU_042 | Upload file and access unauthorized user's file       | Access denied               |
| FU_043 | Direct URL access to uploaded file without permission | Access denied               |
| FU_044 | Upload encrypted file when prohibited                 | Rejected                    |

---

## 6. Performance Test Cases

| TC ID  | Test Scenario                        | Expected Result                       |
|--------|--------------------------------------|---------------------------------------|
| FU_045 | Upload large file under normal load  | Acceptable response time              |
| FU_046 | Upload multiple files simultaneously | System remains stable                 |
| FU_047 | Concurrent uploads by multiple users | No degradation beyond limits          |
| FU_048 | Upload during peak load              | System performs within SLA            |
| FU_049 | Network interruption during upload   | Recovery/retry works correctly        |
| FU_050 | Slow network upload                  | Progress indicator functions properly |

---

## 7. UI/UX Test Cases

| TC ID  | Test Scenario                      | Expected Result             |
|--------|------------------------------------|-----------------------------|
| FU_051 | Verify upload button visibility    | Button displayed correctly  |
| FU_052 | Verify file type information shown | Supported formats displayed |
| FU_053 | Verify upload progress bar         | Progress updates accurately |

| <b>TC ID</b> | <b>Test Scenario</b>                    | <b>Expected Result</b>     |
|--------------|-----------------------------------------|----------------------------|
| FU_054       | Verify success message after upload     | Correct message displayed  |
| FU_055       | Verify error message for invalid upload | Meaningful error displayed |
| FU_056       | Verify uploaded file preview            | Preview shown if supported |
| FU_057       | Verify uploaded file download link      | File downloadable          |
| FU_058       | Verify delete icon for uploaded file    | File removed successfully  |

---

## 8. Compatibility Test Cases

| <b>TC ID</b> | <b>Test Scenario</b>          | <b>Expected Result</b> |
|--------------|-------------------------------|------------------------|
| FU_059       | Upload file from Chrome       | Works correctly        |
| FU_060       | Upload file from Firefox      | Works correctly        |
| FU_061       | Upload file from Safari       | Works correctly        |
| FU_062       | Upload file from Edge         | Works correctly        |
| FU_063       | Upload file on Android device | Works correctly        |
| FU_064       | Upload file on iOS device     | Works correctly        |

| TC ID | Test Scenario | Expected Result |
|-------|---------------|-----------------|
|-------|---------------|-----------------|

|        |                           |                 |
|--------|---------------------------|-----------------|
| FU_065 | Upload from tablet device | Works correctly |
|--------|---------------------------|-----------------|

---

## 9. API-Level Test Cases (if upload uses API)

| TC ID | Test Scenario | Expected Result |
|-------|---------------|-----------------|
|-------|---------------|-----------------|

|        |                                        |                   |
|--------|----------------------------------------|-------------------|
| FU_066 | Upload with valid authentication token | Upload successful |
|--------|----------------------------------------|-------------------|

|        |                               |               |
|--------|-------------------------------|---------------|
| FU_067 | Upload without authentication | Access denied |
|--------|-------------------------------|---------------|

|        |                           |                       |
|--------|---------------------------|-----------------------|
| FU_068 | Upload with expired token | Unauthorized response |
|--------|---------------------------|-----------------------|

|        |                          |                       |
|--------|--------------------------|-----------------------|
| FU_069 | Upload malformed request | Proper error response |
|--------|--------------------------|-----------------------|

|        |                                |                           |
|--------|--------------------------------|---------------------------|
| FU_070 | Verify upload response payload | Correct metadata returned |
|--------|--------------------------------|---------------------------|

|        |                                |                         |
|--------|--------------------------------|-------------------------|
| FU_071 | Verify file checksum integrity | Uploaded file unchanged |
|--------|--------------------------------|-------------------------|

|        |                                |                          |
|--------|--------------------------------|--------------------------|
| FU_072 | Verify storage path generation | Path generated correctly |
|--------|--------------------------------|--------------------------|

---

## 10. Negative Test Cases

| TC ID | Test Scenario | Expected Result |
|-------|---------------|-----------------|
|-------|---------------|-----------------|

|        |                                     |                              |
|--------|-------------------------------------|------------------------------|
| FU_073 | Click upload without selecting file | Validation message displayed |
|--------|-------------------------------------|------------------------------|



| TC ID  | Test Scenario                                   | Expected Result            |
|--------|-------------------------------------------------|----------------------------|
| FU_074 | Upload unsupported and supported files together | Invalid files rejected     |
| FU_075 | Disconnect internet during upload               | Error/retry option shown   |
| FU_076 | Upload same file repeatedly                     | System handles duplicates  |
| FU_077 | Upload file after session timeout               | User prompted to re-login  |
| FU_078 | Submit form before upload completes             | Proper validation enforced |
| FU_079 | Upload file with reserved system filename       | Handled safely             |
| FU_080 | Upload huge number of files at once             | Limit validation triggered |

### What's missing

- 1.File type
- 2.Source and Destination Location
- 3.Application name and module
- 4.Testing type
- 5.Role is missing

## 7.Output type, examples and tone

### ICEPOT Framework:

As a Senior QA Engineer, I need Selenium automation test scripts for the File Upload module of the "XYZ" application. Develop automation test cases using Selenium WebDriver, Java, TestNG, POM, Data-Driven Framework, Follow industry-standard QA automation best practices and coding standards. File should be .xlsx .pdf, size is 25 MB, Include both Positive and Negative automation test cases. Validate all boundary conditions. Validate blank/null scenarios, multiple file section.

### Output:

#### XYZ-Automation

```
|
|
|  └─ src
|
|    └─ main
|
|      └─ java
|
|        └─ base
|
|          └─ BasePage.java
|
|            └─ DriverFactory.java
|
|              └─
|
|                └─ pages
|
|                  └─ LoginPage.java
|
|                    └─ UploadPage.java
```

```
| | | |
| | | |└─ utils
| | | |    ExcelReader.java
| | | |    ConfigReader.java
| | | |    WaitUtil.java
| | | |
| | | |└─ constants
| |
| └─ test
|   java
|     UploadTest.java
|     UploadNegativeTest.java
|     UploadBoundaryTest.java
|
| └─ resources
|     UploadTestData.xlsx
|     config.properties
|
| └─ pom.xml
```

---

**UploadPage.java**

```
package pages;
```

```
import java.nio.file.Paths;
```

```
import java.time.Duration;
```

```
import java.util.List;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.WebElement;
```

```
import org.openqa.selenium.support.ui.ExpectedConditions;
```

```
import org.openqa.selenium.support.ui.WebDriverWait;
```

```
public class UploadPage {
```

```
    private WebDriver driver;
```

```
    private WebDriverWait wait;
```

```
    public UploadPage(WebDriver driver) {
```

```
        this.driver = driver;
```

```
        wait = new WebDriverWait(driver, Duration.ofSeconds(20));
```

```
    }
```

```
private By uploadInput = By.xpath("//input[@type='file']");  
private By uploadButton = By.id("uploadBtn");  
private By successMessage = By.id("successMessage");  
private By errorMessage = By.id("errorMessage");  
private By uploadedFiles = By.cssSelector(".uploaded-file");  
private By deleteButton = By.cssSelector(".delete-file");
```

```
public void uploadFile(String filePath) {
```

```
    driver.findElement(uploadInput)
```

```
.sendKeys(Paths.get(filePath).toAbsolutePath().toString());
```

```
}
```

```
public void uploadMultipleFiles(List<String> files) {
```

```
    StringBuilder paths = new StringBuilder();
```

```
    for(String file : files){
```

```
        paths.append(Paths.get(file).toAbsolutePath()).append("\n");
```

```
}
```

```
    driver.findElement(uploadInput).sendKeys(paths.toString());
```

```
}
```

```
public void clickUpload() {
```

```
    driver.findElement(uploadButton).click();
```

```
}
```

```
public String getSuccessMessage() {
```

```
    wait.until(ExpectedConditions.visibilityOfElementLocated(succ  
essMessage));
```

```
    return driver.findElement(successMessage).getText();
```

```
}
```

```
public String getErrorMessage() {
```

```
    wait.until(ExpectedConditions.visibilityOfElementLocated(error  
Message));
```

```

        return driver.findElement(errorMessage).getText();
    }

    public int uploadedFileCount() {
        return driver.findElements(uploadedFiles).size();
    }

    public void deleteUploadedFile() {
        driver.findElement(deleteButton).click();
    }
}

```

---

## Excel Data

| TestCase | FileName      | Expected |
|----------|---------------|----------|
| TC001    | Sample.pdf    | PASS     |
| TC002    | Sample.xlsx   | PASS     |
| TC003    | Large26MB.pdf | FAIL     |
| TC004    | Image.png     | FAIL     |
| TC005    | Empty.pdf     | FAIL     |

| TestCase | FileName | Expected |
|----------|----------|----------|
|----------|----------|----------|

|       |       |      |
|-------|-------|------|
| TC006 | Blank | FAIL |
|-------|-------|------|

---

**Excel Data Provider**

**@DataProvider(name="uploadData")**

**public Object[][] uploadData(){**

**return ExcelReader.getData("UploadTestData.xlsx","Upload");**

**}**

---

**Positive Upload Test**

**@Test(dataProvider="uploadData")**

**public void verifyValidUpload(String tc,**

**String fileName,**

**String expected){**

**if(!expected.equalsIgnoreCase("PASS"))**

**return;**

**UploadPage page = new UploadPage(driver);**



```
page.uploadFile("TestFiles/" + fileName);
```

```
page.clickUpload();
```

```
Assert.assertEquals(  
    page.getSuccessMessage(),  
    "File uploaded successfully."  
);  
}
```

---

## **Invalid File Type**

**@Test**

```
public void verifyInvalidFileType(){
```

```
UploadPage page = new UploadPage(driver);
```

```
page.uploadFile("TestFiles/Image.png");
```

```
page.clickUpload();
```

```
Assert.assertEquals(  
    page.getErrorMessage(),  
    "Only PDF and XLSX files are allowed."  
);  
  
}
```

---

## PDF Larger Than 25 MB

@Test

```
public void verifyLargePdf(){  
  
    UploadPage page = new UploadPage(driver);  
  
    page.uploadFile("TestFiles/Large26MB.pdf");  
  
    page.clickUpload();  
  
    Assert.assertEquals(  
        page.getErrorMessage(),
```

**"Maximum file size allowed is 25 MB."**

**);**

**}**

---

**XLSX Larger Than 25 MB**

**@Test**

**public void verifyLargeExcel(){**

**UploadPage page = new UploadPage(driver);**

**page.uploadFile("TestFiles/LargeExcel26MB.xlsx");**

**page.clickUpload();**

**Assert.assertEquals(**

**page.getErrorMessage(),**

**"Maximum file size allowed is 25 MB."**

**);**

**}**

---

## **Boundary Test (Exactly 25 MB)**

**@Test**

**public void verify25MBFile(){**

**UploadPage page = new UploadPage(driver);**

**page.uploadFile("TestFiles/PDF25MB.pdf");**

**page.clickUpload();**

**Assert.assertEquals(**

**page.getSuccessMessage(),**

**"File uploaded successfully."**

**);**

**}**

---

## **Boundary Test (25.01 MB)**

**@Test**

**public void verify25Point01MB(){**

```
UploadPage page = new UploadPage(driver);
```

```
page.uploadFile("TestFiles/PDF25_01MB.pdf");
```

```
page.clickUpload();
```

```
Assert.assertTrue(  
    page.getErrorMessage()  
    .contains("25 MB"));
```

```
}
```

---

**Empty File Test**

**@Test**

```
public void verifyEmptyFile(){
```

```
UploadPage page = new UploadPage(driver);
```

```
page.uploadFile("TestFiles/Empty.pdf");
```

```
page.clickUpload();
```

```
Assert.assertEquals(  
    page.getErrorMessage(),  
    "Uploaded file cannot be empty."  
);  
  
}
```

---

## Blank Selection

@Test

```
public void verifyNoFileSelected(){
```

```
    UploadPage page = new UploadPage(driver);
```

```
    page.clickUpload();
```

```
    Assert.assertEquals(  
        page.getErrorMessage(),  
        "Please select a file."
```

```
);  
  
}
```

---

## Multiple PDF Upload

@Test

```
public void verifyMultiplePDFUpload(){
```

```
    UploadPage page = new UploadPage(driver);
```

```
    List<String> files = Arrays.asList(  
        "TestFiles/File1.pdf",  
        "TestFiles/File2.pdf"
```

```
    );
```

```
);
```

```
page.uploadMultipleFiles(files);
```

```
page.clickUpload();
```

```
Assert.assertEquals(  
    "The number of files uploaded is not 2",  
    page.getNumberOfFiles(),  
    2);  
}
```

```
        page.uploadedFileCount(),  
        2  
    );  
  
}
```

---

## PDF + XLSX Upload

@Test

```
public void verifyPdfAndExcel(){  
  
    UploadPage page = new UploadPage(driver);  
  
    List<String> files = Arrays.asList(  
        "TestFiles/Sample.pdf",  
        "TestFiles/Sample.xlsx"  
    );  
  
    page.uploadMultipleFiles(files);  
  
    page.clickUpload();  
}
```



```
Assert.assertEquals(  
    page.uploadedFileCount(),  
    2  
);  
  
}
```

---

## PDF + PNG Upload

@Test

```
public void verifyMixedFiles(){  
  
    UploadPage page = new UploadPage(driver);  
  
    List<String> files = Arrays.asList(  
        "TestFiles/Sample.pdf",  
        "TestFiles/Image.png"  
    );  
  
    page.uploadMultipleFiles(files);  
}
```

```
page.clickUpload();
```

```
Assert.assertEquals(  
    page.getErrorMessage(),  
    "Only PDF and XLSX files are allowed."  
);  
  
}
```

---

## Duplicate File Upload

@Test

```
public void verifyDuplicateUpload(){
```

```
    UploadPage page = new UploadPage(driver);
```

```
    List<String> files = Arrays.asList(  
        "TestFiles/Sample.pdf",  
        "TestFiles/Sample.pdf"  
    );
```

```
page.uploadMultipleFiles(files);
```

```
page.clickUpload();
```

```
Assert.assertEquals(  
    page.getErrorMessage(),  
    "Duplicate files are not allowed."  
);
```

```
}
```

---

**Delete Uploaded File**

**@Test**

```
public void verifyDeleteUploadedFile(){
```

```
    UploadPage page = new UploadPage(driver);
```

```
    page.uploadFile("TestFiles/Sample.pdf");
```

```
page.clickUpload();
```

```
page.deleteUploadedFile();
```

```
Assert.assertEquals(  
    page.uploadedFileCount(),  
    0  
);  
  
}
```

---

## Recommended TestNG Suite

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
```

```
<suite name="XYZ Upload Suite" parallel="methods" thread-  
count="4">
```

```
<test name="Upload Tests">
```

```
<classes>
```

**<class name="tests.UploadTest"/>**

**<class name="tests.UploadNegativeTest"/>**

**<class name="tests.UploadBoundaryTest"/>**

**</classes>**

**</test>**

**</suite>**

**Compare both outputs side by side and write 3 observations on what improved and why.**

- The ICEPOT framework provides comprehensive test scripts along with the required test data for execution.
- It includes positive and negative test cases, covering file validation scenarios.
- The test scripts are execution-ready and can be used directly for testing with minimal or no additional modifications

---

---

---

---

---

---

---

## Scenario 07

### Dashboard / Report Generation

X

"Test the reports section."

Output:

#### Reports Module Test Cases

| TC ID   | Test Scenario                                | Expected Result                                        | Priority |
|---------|----------------------------------------------|--------------------------------------------------------|----------|
| REP_001 | Open Reports page                            | Reports page loads successfully                        | High     |
| REP_002 | Verify report list is displayed              | All available reports are listed                       | High     |
| REP_003 | Generate report with valid filters           | Report is generated correctly                          | High     |
| REP_004 | Generate report without filters (if allowed) | Report generated with default data or validation shown | High     |

| TC ID   | Test Scenario                   | Expected Result                                      | Priority |
|---------|---------------------------------|------------------------------------------------------|----------|
| REP_005 | Apply date range filter         | Only records within the selected range are displayed | High     |
| REP_006 | Apply multiple filters          | Results match all selected criteria                  | High     |
| REP_007 | Reset/Clear filters             | All filters are cleared and default view restored    | Medium   |
| REP_008 | Export report to PDF            | PDF downloads successfully with correct content      | High     |
| REP_009 | Export report to Excel (.xlsx)  | Excel file downloads successfully with correct data  | High     |
| REP_010 | Verify exported file name       | File name follows naming convention                  | Medium   |
| REP_011 | Verify exported data matches UI | Exported data matches displayed report               | High     |

| TC ID   | Test Scenario             | Expected Result                                    | Priority |
|---------|---------------------------|----------------------------------------------------|----------|
| REP_012 | Sort report columns       | Data sorts correctly in ascending/descending order | Medium   |
| REP_013 | Search within report      | Matching records are displayed                     | Medium   |
| REP_014 | Verify pagination         | Navigation between pages works correctly           | Medium   |
| REP_015 | Verify total record count | Record count matches displayed data                | High     |

### Negative Test Cases

| TC ID   | Test Scenario                  | Expected Result                      |
|---------|--------------------------------|--------------------------------------|
| REP_016 | Invalid date range (From > To) | Validation message displayed         |
| REP_017 | Required filter left blank     | Appropriate validation displayed     |
| REP_018 | No data matches filters        | "No records found" message displayed |



| TC ID   | Test Scenario                     | Expected Result                                         |
|---------|-----------------------------------|---------------------------------------------------------|
| REP_019 | Export without generated report   | Appropriate validation or disabled export option        |
| REP_020 | Unauthorized user accesses report | Access denied                                           |
| REP_021 | Session expires before export     | User redirected to login or prompted to re-authenticate |

### Boundary Test Cases

| TC ID   | Scenario                     | Expected Result                             |
|---------|------------------------------|---------------------------------------------|
| REP_022 | Single-day date range        | Correct results returned                    |
| REP_023 | Maximum supported date range | Report generated successfully within limits |
| REP_024 | Large dataset                | Report generated without errors             |
| REP_025 | Maximum number of records    | Pagination and export function correctly    |

### Data Validation

- Verify report totals equal the sum of displayed records.
- Verify calculations (totals, averages, percentages) are correct.

- Verify date/time formats.
- Verify currency and number formats.
- Verify duplicate records are not displayed unless expected.
- Verify data matches the underlying database or API response.

### **Security Test Cases**

- Verify users can only view authorized reports.
- Verify role-based access restrictions.
- Verify exported reports do not expose restricted fields.
- Verify direct URL access to restricted reports is blocked.
- Verify input fields are protected against SQL injection and XSS.

### **Performance Test Cases**

- Report generation completes within the defined SLA.
- Export to PDF and Excel completes within the expected time.
- Large reports do not cause browser crashes or excessive memory usage.
- Multiple users can generate reports concurrently without failures.

### **Automation Scenarios (Selenium)**

These scenarios are well-suited for Selenium automation:

- Generate report using valid filters.
- Validate report table contents.
- Verify sorting.
- Verify searching.
- Verify pagination.
- Verify PDF export initiation.
- Verify Excel export initiation.
- Verify downloaded filename.
- Verify "No records found" message.

- Verify filter reset functionality.
- Verify validation messages for invalid input.
- Verify role-based visibility of reports (with appropriate test accounts).

### **What's missing**

1. Missing Location
2. File type and size
3. If supporting multiple files
4. If it is downloadable or not
5. Testing types
6. Application
7. Roles, Access

### **ICEPOT Framework:**

Act as a Senior QA Engineer. Generate 2 functional test cases for the "XYZ" application covering file-related features such as file upload, download, save, and open operations. Include test cases for file size validation, supported and unsupported file types, multiple file uploads, corrupted files, blank files scenarios. Generate both positive and negative functional test cases, excluding performance test cases. output in an Excel format with columns such as S.No, Module, Feature, Test Scenario, Test Case, Preconditions, Test Steps, Test Data, Expected Result, Priority, and Test Type.

| S. No | Module          | Feature     | Test Scenario                                                                                                 | Test Case                                                                                                                         | Preconditions                                             | Test Steps                                                                                                                                    | Test Data                                                                                                                                                   | Expected Result                                                                                                        | Priority | Test Type           |
|-------|-----------------|-------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|----------|---------------------|
| 1     | File Management | File Upload | Validate file upload functionality for supported, unsupported, boundary, blank, corrupted, and multiple files | Verify that the application function correctly supports uploaded files (.pdf, .xlsx) up to <b>25 MB</b> , reject unsupported file | User is logged in and has access to the File Upload page. | 1. Navigate to the <b>File Upload</b> page.<br>2. Upload a valid <b>PDF</b> (10 MB).<br>3. Upload a valid <b>XLSX</b> (25 MB).<br>4. Upload a | Sample.pdf (10 MB), Report.xlsx (25 MB), Large26 MB.pdf (26 MB), Image.png, Empty.pdf (0 KB), Corrupt.pdf, Sample.pdf + Report.xlsx, Sample.pdf + Image.png | Valid PDF/XLSX files (≤25 MB) uploaded successfully. Files larger than 25 MB, unsupported file types, blank files, and | High     | Positive & Negative |

| S. No | Module | Feature | Test Scenario | Test Case                                                                                                                  | Preconditions | Test Steps                                                                                                                                                                             | Test Data | Expected Result                                                                                                                                                                               | Priority | Test Type |
|-------|--------|---------|---------------|----------------------------------------------------------------------------------------------------------------------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----------|
|       |        |         |               | types, files larger than <b>25 MB</b> , blank (0 KB) and corrupted files, and validates multiple file uploads according to |               | <b>MB PDF.5</b> .<br>Uploa<br>d an<br>unsup<br>porte<br>d<br><b>PNG</b><br>file.6.<br>Uploa<br>d a <b>0 KB</b><br>PDF.7.<br>Uploa<br>d a <b>corrupted PDF.8</b> .<br>Uploa<br>d multip |           | corru<br>pted<br>files<br>are<br>reject<br>ed<br>with<br>appro<br>priate<br>valida<br>tion<br>mess<br>ages.<br>Multi<br>ple<br>valid<br>files<br>uploa<br>d<br>succe<br>ssfull<br>y,<br>while |          |           |

| S. No | Module | Feature | Test Scenario | Test Case       | Preconditions | Test Steps                                                                                                    | Test Data | Expected Result                                                      | Priority | Test Type |
|-------|--------|---------|---------------|-----------------|---------------|---------------------------------------------------------------------------------------------------------------|-----------|----------------------------------------------------------------------|----------|-----------|
|       |        |         |               | business rules. |               | le valid files (PDF + XLSX).<br>9. Upload multiple files containing one valid PDF and one unsupported PNG.10. |           | mixed valid/invalid uploads are handled according to business rules. |          |           |

| S. No | Module          | Feature                     | Test Scenario                                           | Test Case                                                     | Preconditions                                                   | Test Steps                                                         | Test Data                                                      | Expected Result                                        | Priority | Test Type           |
|-------|-----------------|-----------------------------|---------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------|--------------------------------------------------------|----------|---------------------|
|       |                 |                             |                                                         |                                                               |                                                                 | Verify the validation messages and uploaded file list.             |                                                                |                                                        |          |                     |
| 2     | File Management | File Download / Save / Open | Validate download, save, and open operations for upload | Verify that uploaded supported files can be saved, downloaded | User is logged in and at least one valid PDF/XLSX file has been | 1. Upload a valid PDF or XLSX file.2. Click <b>Save</b> and verify | Sample.pdf, Report.xlsx, Corrupt.pdf, Deleted/Unavailable file | Valid files are saved successfully, downloaded without | High     | Positive & Negative |

| S. No | Module | Feature | Test Scenario | Test Case                                                                                     | Preconditions        | Test Steps                                                                                                            | Test Data | Expected Result                                                                                                   | Priority | Test Type |
|-------|--------|---------|---------------|-----------------------------------------------------------------------------------------------|----------------------|-----------------------------------------------------------------------------------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------|----------|-----------|
|       |        |         | ded files     | oaded, and opened successfully, and that invalid or unavailable files are handled gracefully. | upload successfully. | the confirmation message.3. Download the uploaded file.4. Open the downloaded file and verify its contents.5. Attempt |           | corruption, and opened correctly. Attempts to download or open unavailable or corrupted files display appropriate |          |           |



| S. No | Module | Feature | Test Scenario | Test Case | Preconditions | Test Steps                                                                                                                                                                                         | Test Data | Expected Result                                                                             | Priority | Test Type |
|-------|--------|---------|---------------|-----------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|---------------------------------------------------------------------------------------------|----------|-----------|
|       |        |         |               |           |               | pt to<br>downl<br>oad or<br>open<br>a<br>delete<br>d,<br>corru<br>pted,<br>or<br>unava<br>ilable<br>file (if<br>applic<br>able).<br>6.<br>Verify<br>the<br>displa<br>yed<br>error<br>mess<br>ages. |           | error<br>mess<br>ages<br>witho<br>ut<br>causi<br>ng<br>appli<br>catio<br>n<br>failur<br>es. |          |           |

Compare both outputs side by side and **write 3 observations** on what improved and why.

- The ICEPOT framework prompt covers comprehensive functional test cases, including edge cases and exceptional scenarios, whereas a weak prompt generates only generic test cases.
- The test cases generated using the ICEPOT framework are well-structured, easy to understand, and user-friendly.
- The test cases are execution-ready, allowing QA engineers to begin testing with minimal or no additional modifications.